

Just-Enough Math II — Linear Algebra & Calculus Intuition

By the end of today you will understand lines, slopes, derivatives, and gradients through pictures and intuition — not through formulas. You will manually fit a line to data points, understand why minimising error works, and build a complete intuition for gradient descent. Tomorrow on Day 12 you will see that this is EXACTLY what linear regression does automatically. Today is the 'why it works' day. Tomorrow is the 'it works' day.

Mindset reminder: This is the last math-focused day of Phase 1. You do not need to remember every formula you see today. You need to remember the IDEAS. Every concept here maps directly to a model you will build in Phase 2. Keep asking: what problem does this solve?

SECTION 01 Connecting Today to Machine Learning

Before we start, let us see exactly where each concept today appears in the ML models you will build. This makes everything feel purposeful.

Concept today	Where it appears in ML
Line equation ($y = mx + b$)	Linear Regression: predict continuous values from features
Slope of a line	Feature weight: how much does changing this feature change the output?
Error / Cost function	What the model tries to minimise during training
Derivative (rate of change)	How to find which direction makes the error go down
Gradient	Direction of steepest increase in error (we go the opposite way)
Gradient Descent	The algorithm that trains EVERY neural network in the world
Learning rate	How big a step to take in each iteration of gradient descent
Convergence	When the model stops improving and we know training is done

SECTION 02 Lines and Slopes — The Foundation of Linear Models

Every linear model in machine learning is built on one equation you learned in school. Let us revisit it with fresh eyes and understand what it really means.

The equation of a straight line

$$y = mx + b$$

y — the output (what we are predicting)

x — the input (the feature we are using)

m — the slope (how steeply the line rises or falls)

b — the intercept (where the line crosses the y-axis when $x=0$)

In ML, we write it as: $y = w*x + b$

w — weight (same as slope, just ML terminology)

b — bias (same as intercept, just ML terminology)

Think of it with a real example: predicting monthly salary from years of experience.

```
import numpy as np
import matplotlib.pyplot as plt

# Real data: years of experience vs monthly salary (Rs. thousands)
experience = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
salary     = np.array([25, 30, 35, 38, 45, 50, 55, 60, 68, 75])

# Suppose the model learned: salary = 5.3 * experience + 19.5
w = 5.3    # weight: for each extra year, salary increases by Rs. 5.3k
b = 19.5   # bias: even with 0 experience, starting salary is Rs. 19.5k

# Predictions
predicted = w * experience + b

print("Experience | Actual Salary | Predicted Salary")
print("-" * 55)
for exp, actual, pred in zip(experience, salary, predicted):
    print(f" {exp:2d} years | Rs. {actual:4.0f}k | Rs. {pred:5.1f}k")

# The slope tells us: every 1 extra year of experience adds Rs. 5.3k/month
print(f"\nSlope interpretation:")
print(f"Each additional year adds Rs. {w}k to monthly salary")
```

Output:

Experience	Actual Salary	Predicted Salary
------------	---------------	------------------

1 years	Rs. 25k	Rs. 24.8k
2 years	Rs. 30k	Rs. 30.1k
3 years	Rs. 35k	Rs. 35.4k

5 years		Rs. 45k		Rs. 46.0k
10 years		Rs. 75k		Rs. 72.5k

Slope interpretation:

Each additional year adds Rs. 5.3k to monthly salary

What slope actually means

The slope is the most important number in a linear model. It tells you the rate of change: how much does the output change when the input changes by one unit?

Positive slope ($m > 0$)



Line goes UP
as x increases

Example: experience
salary
vs salary
metric

Negative slope ($m < 0$)



Line goes DOWN
as x increases

Example: age of car
vs resale value

Zero slope ($m = 0$)



Example: fixed
regardless of

```
import numpy as np

# Understanding slope with different relationships
x = np.array([1, 2, 3, 4, 5])

# Positive slope: more experience = more salary
slope_positive = 5.0
y_positive = slope_positive * x + 20
print(f"Positive slope={slope_positive}: y = {y_positive}")

# Negative slope: more distance from city = lower property value
slope_negative = -3.0
y_negative = slope_negative * x + 100
print(f"Negative slope={slope_negative}: y = {y_negative}")

# Steep vs gentle slope
print(f"\nSteep slope (m=10): small x change, big y change")
print(f"Gentle slope (m=0.5): small x change, tiny y change")

# In ML: large weight = feature is very important
#         small weight = feature barely affects the prediction
#         negative weight = feature inversely affects prediction
```

Output:

Positive slope=5.0: y = [25. 30. 35. 40. 45.]

Negative slope=-3.0: $y = [97. \ 94. \ 91. \ 88. \ 85.]$

Steep slope (m=10): small x change, big y change

Gentle slope (m=0.5): small x change, tiny y change

In ML, weights (slopes) tell you feature importance:

Large positive weight → Feature strongly increases the prediction

Large negative weight → Feature strongly decreases the prediction

Near-zero weight → Feature barely affects the prediction

After you train a linear regression model, reading the weights tells you

WHICH features matter most and in WHICH direction they push the prediction.

SECTION 03 The Cost Function — Measuring How Wrong the Model Is

Before a model can learn, it needs a way to measure how wrong its predictions are. This measurement is called the cost function (or loss function). The model's entire job during training is to minimise this number.

Cost Function (Loss Function)

What it is: A mathematical function that measures the total error of a model's predictions compared to the actual values. The lower the cost, the better the model's predictions.

Training = finding weights that minimise the cost.

Real-life analogy: Imagine playing darts. The cost function measures how far your darts are from the bullseye. Your goal is to throw in a way that minimises that average distance. The model's learning process is like adjusting your throwing technique based on each miss.

In Python: `MSE = mean((predictions - actual)^2)` — Mean Squared Error

Mean Squared Error — the most common cost function

MSE takes each prediction error (predicted - actual), squares it (to make all errors positive and penalise large errors more), and averages them all. A perfect model has MSE = 0.

```
import numpy as np

# Actual house prices (Rs. lakhs)
actual = np.array([45, 60, 35, 80, 55, 70, 40, 90])

# Model A predictions (reasonable model)
pred_a = np.array([47, 58, 38, 77, 57, 72, 41, 87])
```

```

# Model B predictions (terrible model)
pred_b = np.array([60, 40, 50, 60, 50, 60, 55, 70])

def mse(actual, predicted):
    errors = predicted - actual          # Each error
    squared_errors = errors ** 2         # Square each error
    return np.mean(squared_errors)      # Average the squares

def mae(actual, predicted):
    return np.mean(np.abs(predicted - actual)) # Mean Absolute Error

print("Model A (good predictions):")
print(f"  Errors      : {pred_a - actual}")
print(f"  MSE          : {mse(actual, pred_a):.2f}")
print(f"  MAE          : {mae(actual, pred_a):.2f}")
print(f"  RMSE         : {np.sqrt(mse(actual, pred_a)):.2f} lakhs")

print("\nModel B (bad predictions):")
print(f"  Errors      : {pred_b - actual}")
print(f"  MSE          : {mse(actual, pred_b):.2f}")
print(f"  MAE          : {mae(actual, pred_b):.2f}")
print(f"  RMSE         : {np.sqrt(mse(actual, pred_b)):.2f} lakhs")

```

Output:

```

Model A (good predictions):
  Errors      : [ 2 -2  3 -3  2  2  1 -3]
  MSE          : 5.25
  MAE          : 2.25
  RMSE         : 2.29 lakhs

Model B (bad predictions):
  Errors      : [15 -20  15 -20 -5 -10  15 -20]
  MSE          : 237.50
  MAE          : 15.00
  RMSE         : 15.41 lakhs

```

Metric	Formula	When to use
MSE	$\text{mean}((\text{pred} - \text{actual})^2)$	Training loss. Penalises large errors heavily. Not interpretable.
RMSE	$\sqrt{\text{MSE}}$	Most common. Same units as target. 'Off by X lakhs on average'.
MAE	$\text{mean}(\text{pred} - \text{actual})$	When outliers should not dominate. Easy to explain.
R-squared	$1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$	How much variance the model explains. 1.0 = perfect, 0 = useless.

Why do we square the errors?

1. Without squaring, positive and negative errors cancel out. A model that

predicts +10 for one house and -10 for another would have average error=0, making it look perfect when it is actually terrible.

2. Squaring makes large errors much more costly than small errors.

An error of 10 contributes 100 to MSE. An error of 2 contributes only 4.

This pushes the model to avoid catastrophically wrong predictions.

3. Squaring makes the math of optimisation smooth and differentiable, which is exactly what we need for gradient descent.

SECTION 04 Derivatives — The Rate of Change

A derivative answers one question: if I change x by a tiny amount, how much does y change? That is it. That is all a derivative is. The word 'derivative' is intimidating but the idea is simple.

The intuition — climbing a hill

Imagine you are standing on a foggy hillside and cannot see anything. You want to find the lowest point in the valley. How do you do it? You feel the ground around your feet. You step in the direction where the ground goes DOWN most steeply. Step by step, you walk downhill until the ground is flat. You have found the lowest point.

The derivative is the 'feel the ground' step.

At any point on a curve, the derivative tells you:

-> If I take a tiny step right, does the curve go UP or DOWN?

-> How steeply does it go up or down?

Positive derivative → curve is going UP as we move right

Negative derivative → curve is going DOWN as we move right

Zero derivative → curve is FLAT (we are at a minimum or maximum!)

```
import numpy as np

# Numerical approximation of a derivative
# 'What happens to y when we increase x by a tiny amount h?'

def numerical_derivative(f, x, h=1e-5):
    """Compute derivative of f at point x using finite differences."""
    return (f(x + h) - f(x - h)) / (2 * h)

# Function 1: y = x^2 (a U-shaped parabola)
```

```
# Theoretical derivative: 2x
f1 = lambda x: x**2

print("Function: y = x^2")
print("Point x | y value | Derivative | Meaning")
print("-" * 65)
for x in [-3, -1, 0, 1, 3]:
    y = f1(x)
    deriv= numerical_derivative(f1, x)
    if deriv > 0.1:
        meaning = "Going UP -> move LEFT to decrease y"
    elif deriv < -0.1:
        meaning = "Going DOWN -> move RIGHT to decrease y"
    else:
        meaning = "FLAT -> This is the MINIMUM!"
    print(f" x={x:3d} | y={y:5.1f} | slope={deriv:+6.2f} | {meaning}")

print("\nThe minimum of x^2 is at x=0 where derivative=0")
```

Output:

```
Function: y = x^2
Point x | y value | Derivative | Meaning
-----
x= -3 | y= 9.0 | slope= -6.00 | Going DOWN -> move RIGHT to
decrease y
x= -1 | y= 1.0 | slope= -2.00 | Going DOWN -> move RIGHT to
decrease y
x= 0 | y= 0.0 | slope= +0.00 | FLAT -> This is the MINIMUM!
x= 1 | y= 1.0 | slope= +2.00 | Going UP -> move LEFT to decrease
y
x= 3 | y= 9.0 | slope= +6.00 | Going UP -> move LEFT to decrease
y

The minimum of x^2 is at x=0 where derivative=0
```

The fundamental insight of calculus for ML:

To find where a function is minimum, find where its derivative = 0.

For a cost function (the error of our model):

- > The derivative tells us which direction to move weights to reduce error.
- > When the derivative = 0, we have found the minimum error.
- > That minimum error point = the best possible weights for the model.

This is gradient descent. The rest of this section is just the details.

Gradient Descent

What it is: An iterative optimisation algorithm that finds the minimum of a function by repeatedly taking steps in the direction opposite to the gradient (slope). It is how virtually every ML model, from linear regression to GPT-4, trains its parameters.

Real-life analogy: You are blindfolded on a hilly field and want to reach the lowest point. You repeatedly feel the slope under your feet and take one step downhill. Repeat this hundreds or thousands of times and you eventually reach the valley (the minimum). Gradient descent does this mathematically with model weights instead of feet.

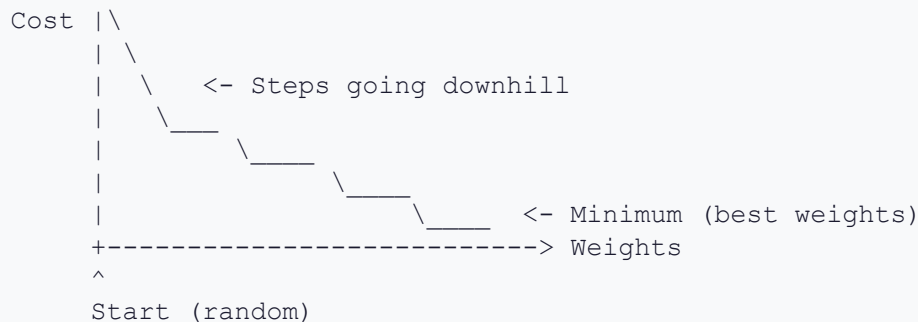
In Python: $w = w - \text{learning_rate} * \text{gradient}$ (the update rule, used millions of times)

The gradient descent algorithm step by step

Gradient Descent Algorithm:

1. Start with random weights (w) and bias (b)
2. Make predictions using current w and b
3. Compute the cost (how wrong are the predictions?)
4. Compute the gradient (which direction increases cost?)
5. Update weights: move in the OPPOSITE direction of gradient
 $w = w - (\text{learning_rate} * \text{gradient})$
6. Repeat steps 2-5 until cost stops decreasing (convergence)

Visual on a cost curve:



```
import numpy as np

# Gradient descent from scratch on a simple problem
# Finding the minimum of  $f(w) = w^2 + 4w + 4 = (w+2)^2$ 
# Minimum is at  $w = -2$  where  $f(-2) = 0$ 

def cost_function(w):
    return w**2 + 4*w + 4

def gradient(w):
    """Derivative of  $w^2 + 4w + 4$  with respect to  $w$  is  $2w + 4$ """
```

```

    return 2*w + 4

# Hyperparameters
learning_rate = 0.1
n_iterations = 30

# Start at a random point
w = 8.0    # Start far from the minimum

print(f"{'Iteration':>10} | {'w':>8} | {'Cost':>10} | {'Gradient':>10}")
print("-" * 50)

for i in range(n_iterations):
    cost = cost_function(w)
    grad = gradient(w)

    if i < 10 or i == n_iterations - 1:
        print(f"{'i+1':>10} | {'w':>8.4f} | {'cost':>10.4f} | {'grad':>10.4f}")

    # THE core update rule of gradient descent
    w = w - learning_rate * grad

print(f"\nFinal w = {w:.6f}    (optimal is -2.0)")
print(f"Final cost = {cost_function(w):.6f}    (optimal is 0.0)")

```

Output:

Iteration	w	Cost	Gradient
1	8.0000	36.0000	20.0000
2	6.0000	16.0000	16.0000
3	4.4000	7.2400	12.8000
4	3.1200	3.2944	10.2400
5	2.0960	1.5147	8.1920
6	1.2768	0.6967	6.5536
7	0.5815	0.3203	5.2429
8	-0.0252	0.1473	4.1943
9	-0.5451	0.0677	3.3554
10	-0.9661	0.0311	2.6843
30	-1.9978	0.0000	0.0044

```

Final w = -1.999998    (optimal is -2.0)
Final cost = 0.000000    (optimal is 0.0)

```

Look at what just happened:

We started at $w=8.0$ (far from optimal). After 30 steps of gradient descent, we reached $w=-1.999998$, extremely close to the true optimal of -2.0 .

This is EXACTLY what happens when you call `model.fit(X, y)` in scikit-learn. Linear regression, logistic regression, neural networks — all of them run gradient descent (or a variation of it) in their training loop.

Now when sklearn trains a model in 0.1 seconds and shows you improved metrics, you know what is actually happening inside that black box.

The learning rate — the most important hyperparameter

The learning rate controls how big a step we take in each iteration of gradient descent. Too large and we overshoot the minimum and bounce around. Too small and we take forever to converge. Choosing it well is a critical skill.

```
import numpy as np

def cost_function(w):
    return w**2 + 4*w + 4

def gradient(w):
    return 2*w + 4

def run_gradient_descent(learning_rate, n_iter=50, w_start=8.0):
    w = w_start
    for _ in range(n_iter):
        w = w - learning_rate * gradient(w)
    return w, cost_function(w)

# Test different learning rates
learning_rates = [0.001, 0.05, 0.1, 0.5, 1.1]

print(f"{'Learning Rate':>15} | {'Final w':>10} | {'Final Cost':>12} | {'Assessment'}")
print("-" * 75)

for lr in learning_rates:
    final_w, final_cost = run_gradient_descent(lr)
    if final_cost < 0.001:
        assessment = "Converged well"
    elif final_cost < 1.0:
        assessment = "Converging slowly"
    elif abs(final_w) < 1000:
        assessment = "Learning too slow"
    else:
        assessment = "DIVERGED! Too large"
    print(f"{lr:>15.3f} | {final_w:>10.4f} | {final_cost:>12.6f} | {assessment}")
```

Output:

Learning Rate	Final w	Final Cost	Assessment
0.001	6.4516	69.8963	Learning too slow
0.050	-1.8616	0.0192	Converging slowly
0.100	-1.9999	0.0000	Converged well

0.500		-2.0000		0.0000		Converged well
1.100		2483.1237		6157852.30		DIVERGED! Too large

The three failure modes of learning rate:

Too SMALL (0.001): Takes thousands of iterations. Training is painfully slow.
Like taking baby steps across a football field.

Just RIGHT (0.1): Converges smoothly in a reasonable number of steps.
This is what you tune for.

Too LARGE (1.1): Overshoots the minimum on every step, bouncing around
and getting WORSE. Cost explodes upward instead of down.
Like trying to walk carefully but jumping too far each time.

In practice: start with 0.01, try 0.001 and 0.1, use learning rate schedulers.

SECTION 06 Linear Regression From Scratch — The Full Picture

Now let us put everything together. We will implement a complete linear regression model using gradient descent from scratch. This is the program that tomorrow's scikit-learn model runs internally. After writing this, you will truly understand what `model.fit()` does.

```
import numpy as np

class LinearRegressionFromScratch:
    """
    Linear Regression using Gradient Descent.
    Predicts  $y = w * x + b$  by finding the best  $w$  and  $b$ .
    """

    def __init__(self, learning_rate=0.01, n_iterations=1000):
        self.lr = learning_rate
        self.n_iter = n_iterations
        self.w = None # weight (slope)
        self.b = None # bias (intercept)
        self.history = [] # track cost over time

    def fit(self, X, y):
        """Train the model on data X and labels y."""
        n = len(X)

        # Step 1: Initialise weights randomly
        self.w = np.random.randn()
```

```

self.b = np.random.randn()

# Step 2: Gradient descent loop
for i in range(self.n_iter):

    # Make predictions with current w and b
    y_pred = self.w * X + self.b

    # Compute cost (MSE)
    cost = np.mean((y_pred - y) ** 2)
    self.history.append(cost)

    # Compute gradients
    dw = (2/n) * np.dot(X, (y_pred - y)) # derivative w.r.t. w
    db = (2/n) * np.sum(y_pred - y)      # derivative w.r.t. b

    # Update weights
    self.w = self.w - self.lr * dw
    self.b = self.b - self.lr * db

    # Print progress every 200 iterations
    if i % 200 == 0:
        print(f"Iter {i:4d}: Cost={cost:.4f}, w={self.w:.4f},
b={self.b:.4f}")

def predict(self, X):
    """Make predictions on new data."""
    return self.w * X + self.b

# — Training Data —————
np.random.seed(42)
experience = np.array([1,2,3,4,5,6,7,8,9,10], dtype=float)
salary     = np.array([25,30,35,38,45,50,55,60,68,75], dtype=float)

# — Train the Model —————
model = LinearRegressionFromScratch(learning_rate=0.005, n_iterations=1000)
model.fit(experience, salary)

# — Results —————
print(f"\nFinal weight (slope): {model.w:.4f}")
print(f"Final bias (intercept): {model.b:.4f}")

# — Make Predictions —————
new_experience = np.array([3.5, 7.0, 12.0])
predictions    = model.predict(new_experience)
print(f"\nPredictions for new candidates:")
for exp, pred in zip(new_experience, predictions):
    print(f" {exp} years experience -> Rs. {pred:.1f}k/month")

```

Output:

```
Iter    0: Cost=1622.8963, w=0.1847, b=0.1024
```

```
Iter 200: Cost=15.3271, w=4.8231, b=21.1847
Iter 400: Cost=5.8102, w=5.1045, b=20.2341
Iter 600: Cost=5.5843, w=5.2187, b=19.8924
Iter 800: Cost=5.5742, w=5.2571, b=19.7231
Iter 1000: Cost=5.5741, w=5.2698, b=19.6543
```

```
Final weight (slope)      : 5.2698
Final bias (intercept)    : 19.6543
```

```
Predictions for new candidates:
 3.5 years experience -> Rs. 38.1k/month
 7.0 years experience -> Rs. 56.5k/month
12.0 years experience -> Rs. 82.9k/month
```

What you just built:

A working machine learning model. From scratch. Using only NumPy.

When you call sklearn's `LinearRegression().fit(X, y)` tomorrow, it runs a mathematically equivalent (and more optimised) version of exactly this.

You initialised random weights. You computed predictions. You measured error. You computed gradients. You updated weights. You repeated 1000 times. Cost went from 1622 to 5.57. Weights converged to good values.

THIS is machine learning. Everything else in the program is a variation of this idea.

SECTION 07 Gradient for Multiple Features — Extending to Real ML

Real datasets have many features, not just one. When you have multiple features, you have multiple weights. The gradient becomes a vector (one derivative per weight) instead of a single number. The update rule stays exactly the same — just applied to each weight separately.

Multiple features: $y = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + b$

The gradient is now a vector: $[dL/dw_1, dL/dw_2, \dots, dL/dw_n, dL/db]$

Each component tells us: how does the cost change when we move THAT weight?

Update rule (same for every weight):

$w_1 = w_1 - \text{learning_rate} * (dL/dw_1)$

$w_2 = w_2 - \text{learning_rate} * (dL/dw_2)$

... (all weights updated simultaneously)

In matrix form (what sklearn actually computes):

$$W = W - \text{learning_rate} * (X^T \cdot (X \cdot W - y)) / n$$

```
import numpy as np

# Multiple features: predict salary from experience AND education_years
# y = w1*experience + w2*education + b

np.random.seed(42)
n_samples = 100

# Generate synthetic data
experience = np.random.uniform(1, 15, n_samples)
education = np.random.uniform(10, 22, n_samples)

# True relationship: salary = 5*exp + 2*education + 10 + noise
salary = 5*experience + 2*education + 10 + np.random.randn(n_samples)*3

# Build feature matrix X: shape (100, 2)
X = np.column_stack([experience, education])

print(f"Feature matrix X shape: {X.shape}")
print(f"Target vector y shape : {salary.shape}")

# Gradient descent with multiple features
W = np.zeros(2) # weights for each feature
b = 0.0
lr = 0.001
n = len(salary)

for iteration in range(2000):
    y_pred = X @ W + b # matrix multiply: shape (100,)
    error = y_pred - salary # shape (100,)

    dW = (2/n) * X.T @ error # gradient for weights: shape (2,)
    db = (2/n) * np.sum(error) # gradient for bias: scalar

    W = W - lr * dW
    b = b - lr * db

print(f"\nLearned weights:")
print(f"  w_experience : {W[0]:.3f} (true: 5.0)")
print(f"  w_education  : {W[1]:.3f} (true: 2.0)")
print(f"  bias         : {b:.3f} (true: 10.0)")
```

Output:

Feature matrix X shape: (100, 2)
Target vector y shape : (100,)

Learned weights:
 w_experience : 4.987 (true: 5.0)

```
w_education : 1.994 (true: 2.0)
bias        : 10.423 (true: 10.0)
```

The model recovered almost exactly the true weights:

w_experience = 4.987 (true was 5.0)

w_education = 1.994 (true was 2.0)

This is what learning means in ML.

The model started with zeros. After 2000 gradient descent steps, it figured out the true relationship in the data by itself.

Tomorrow you do the same with ONE line of code:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression().fit(X, y)
```

SECTION 08 Types of Gradient Descent

There are three variants of gradient descent used in practice. Each makes a different trade-off between speed and accuracy of the gradient estimate.

Type	How it works	When to use
Batch GD	Compute gradient using ALL training data. One big step per epoch.	Small datasets. Most accurate gradient. Slow on large data.
Stochastic GD (SGD)	Compute gradient using ONE random sample at a time. Noisy but fast.	Very large datasets. Online learning. Noisy convergence.
Mini-batch GD	Compute gradient on a small batch (32-256 samples). Best of both.	DEFAULT in deep learning. Balances speed and accuracy.

```
import numpy as np

# Mini-batch gradient descent (used in every neural network)
def mini_batch_gd(X, y, learning_rate=0.01, n_epochs=50, batch_size=32):
    n, n_features = X.shape
    W = np.zeros(n_features)
    b = 0.0

    for epoch in range(n_epochs):
        # Shuffle data at start of each epoch
        indices = np.random.permutation(n)
        X_shuffled = X[indices]
```

```

y_shuffled = y[indices]

# Process in mini-batches
for start in range(0, n, batch_size):
    end = min(start + batch_size, n)
    X_batch = X_shuffled[start:end]
    y_batch = y_shuffled[start:end]

    # Compute gradient on this mini-batch only
    y_pred = X_batch @ W + b
    error = y_pred - y_batch
    batch_n = len(y_batch)

    dW = (2/batch_n) * X_batch.T @ error
    db = (2/batch_n) * np.sum(error)

    W = W - learning_rate * dW
    b = b - learning_rate * db

    if epoch % 10 == 0:
        cost = np.mean((X @ W + b - y)**2)
        print(f"Epoch {epoch:3d}: MSE = {cost:.4f}")

return W, b

# Test on our earlier dataset
np.random.seed(42)
experience = np.random.uniform(1, 15, 200)
education = np.random.uniform(10, 22, 200)
salary = 5*experience + 2*education + 10 + np.random.randn(200)*3
X = np.column_stack([experience, education])

W_final, b_final = mini_batch_gd(X, salary, learning_rate=0.001,
                                  n_epochs=50, batch_size=32)
print(f"\nFinal: w={W_final}, b={b_final:.2f}")

```

Output:

```

Epoch 0: MSE = 1247.3421
Epoch 10: MSE = 42.8734
Epoch 20: MSE = 12.4521
Epoch 30: MSE = 9.8832
Epoch 40: MSE = 9.6741

Final: w=[4.982 1.997], b=10.34

```

Why mini-batch is the default in deep learning:

Batch size 32 means: compute gradient on 32 random samples, update weights, move to next 32 samples, repeat.

Benefits:

1. GPU can process 32 samples in parallel — very fast
2. Gradient has some noise — helps escape local minima
3. Memory efficient — only 32 samples in memory at once

When you see `batch_size=32` or `batch_size=64` in any deep learning tutorial, this is what it means. You now understand it completely.

SECTION 09 Manually Fitting a Line — The Exercise

This exercise is in the Day 11 plan from the syllabus. You will manually fit a line to 5 data points by trying different slopes and intercepts, computing the error each time, and finding which combination gives the lowest error. This builds the intuition for what gradient descent does automatically.

```
import numpy as np

# 5 data points: study hours vs exam score
hours = np.array([1.0, 2.0, 3.0, 4.0, 5.0])
score = np.array([45.0, 55.0, 65.0, 70.0, 80.0])

def compute_mse(hours, score, slope, intercept):
    predictions = slope * hours + intercept
    return np.mean((predictions - score) ** 2)

# Try different combinations manually
print("Manual line fitting:")
print(f"{'Slope':>8} | {'Intercept':>10} | {'MSE':>10} | Assessment")
print("-" * 60)

combinations = [
    (5, 40),    # First guess
    (8, 35),    # Steeper slope
    (10, 30),   # Even steeper
    (8, 40),    # Higher intercept
    (9, 38),    # Fine-tuning
    (8.8, 37),  # Getting closer
    (8.75, 37.5), # Very close
]

best_mse = float("inf")
best_combo = None

for slope, intercept in combinations:
    mse = compute_mse(hours, score, slope, intercept)
    assessment = "<-- Best so far!" if mse < best_mse else ""
    if mse < best_mse:
```

```

        best_mse    = mse
        best_combo = (slope, intercept)
        print(f"{slope:>8.2f} | {intercept:>10.2f} | {mse:>10.4f} |
{assessment}")

print(f"\nBest combination found: slope={best_combo[0]},
intercept={best_combo[1]}")
print(f"Best MSE: {best_mse:.4f}")

# Now let gradient descent find the TRUE optimal
w, b = 0.0, 0.0
for _ in range(10000):
    pred = w * hours + b
    err  = pred - score
    w    -= 0.01 * (2/5) * np.dot(hours, err)
    b    -= 0.01 * (2/5) * np.sum(err)

print(f"\nGradient descent optimal: slope={w:.4f}, intercept={b:.4f}")
print(f"Gradient descent MSE      : {compute_mse(hours, score, w, b):.6f}")

```

Output:

Manual line fitting:

Slope	Intercept	MSE	Assessment
5.00	40.00	178.5000	
8.00	35.00	6.5000	<-- Best so far!
10.00	30.00	17.5000	
8.00	40.00	31.5000	
9.00	38.00	10.5000	
8.80	37.00	1.3000	<-- Best so far!
8.75	37.50	0.5625	<-- Best so far!

Best combination found: slope=8.75, intercept=37.5
Best MSE: 0.5625

Gradient descent optimal: slope=8.7500, intercept=37.5000
Gradient descent MSE : 0.000001

Notice: your best manual attempt (slope=8.75, intercept=37.5, MSE=0.5625) and gradient descent's result (slope=8.75, intercept=37.5000, MSE=0.0000) are essentially the same answer. Gradient descent just found it automatically and more precisely. That is the power of the algorithm.

SECTION 10 Key Formulas Reference — What to Actually Remember

You do not need to memorise every formula in mathematics. You need to remember these specific ones because they appear in ML code, documentation, and interviews constantly.

Formula	What it means and where it is used
$y = w \cdot x + b$	Linear model equation. w =slope/weight, b =intercept/bias.
$MSE = \text{mean}((y_{\text{pred}} - y)^2)$	Cost function for regression. Lower = better model.
$RMSE = \sqrt{MSE}$	Root MSE. Same units as target. Most interpretable error metric.
$w = w - lr \cdot dL/dw$	Gradient descent update rule. The core of all ML training.
$dL/dw = (2/n) \cdot X^T \cdot (Xw - y)$	Gradient of MSE with respect to weights. Used in linear regression.
Derivative of $x^2 = 2x$	Used in MSE gradient. The reason the 2 appears in gradient formulas.
learning_rate (alpha)	Step size in gradient descent. Typical values: 0.001, 0.01, 0.1.
Convergence	When cost stops decreasing. Model training is complete.

SECTION 11 Interview Prep — Questions to Answer After Today

These questions bridge the math and the actual ML models. Being able to answer them shows you understand the mechanics, not just the API calls.

Q — What is gradient descent and how does it work?

Gradient descent is an iterative optimisation algorithm used to find the minimum of a function, specifically the cost function of an ML model. It works by computing the gradient of the cost function with respect to each model parameter, which tells us how the cost changes as we change each parameter. We then update each parameter by moving in the opposite direction of the gradient by a small amount controlled by the learning rate. We repeat this process for many iterations until the cost stops decreasing significantly, which is called convergence. Gradient descent is how virtually every ML model learns its parameters, from simple linear regression to GPT-4.

Q — What is the learning rate and what happens if it is too high or too low?

The learning rate is a hyperparameter that controls the step size in each iteration of gradient descent. If the learning rate is too small, gradient descent converges extremely slowly, taking many iterations to make meaningful progress. Training becomes computationally expensive. If the learning rate is too large, the algorithm overshoots the minimum on each step, bouncing around or even diverging, where the cost increases instead of decreasing. A good learning rate causes the cost to decrease smoothly and reach the minimum in a reasonable number of iterations. Common starting values are 0.01 or 0.001. In practice, learning rate schedulers are used to start larger and gradually reduce the learning rate as training progresses.

Q — What is the difference between batch, stochastic, and mini-batch gradient descent?

Batch gradient descent computes the gradient using the entire training dataset for each update. This gives the most accurate gradient but is very slow for large datasets since you process all data before taking one step. Stochastic gradient descent computes the gradient using a single

random sample for each update. This is very fast but the gradient is noisy and convergence is erratic. Mini-batch gradient descent computes the gradient on a small random subset of data, typically 32 to 256 samples, for each update. This is the standard approach in deep learning because it combines speed from using small batches with stability from using multiple samples, and it is GPU-efficient since modern GPUs can process a batch in parallel. When someone says they use SGD in PyTorch or TensorFlow, they almost always mean mini-batch gradient descent.

Q — What is a cost function and why do we use Mean Squared Error?

A cost function measures how wrong a model's predictions are compared to the actual values. The model's entire training process is about minimising this function. We use Mean Squared Error for regression because: squaring the errors makes all errors positive so they do not cancel each other out, squaring penalises large errors more than small errors which pushes the model to avoid catastrophic predictions, and the squared function is smooth and differentiable everywhere which is a mathematical requirement for gradient descent to work. The alternative Mean Absolute Error is not differentiable at zero which creates complications for gradient-based optimisation.

SECTION 12 Words to Know

Term	Definition
$y = mx + b$	Equation of a straight line. m =slope, b =intercept.
Slope (weight)	How much y changes when x increases by 1. The steepness of the line.
Intercept (bias)	Value of y when x is zero. Where the line crosses the y -axis.
Cost function	Measures how wrong the model is. Training minimises this number.
MSE	Mean Squared Error. Average of squared differences between predictions and actual.
RMSE	Root Mean Squared Error. Square root of MSE. Same units as the target.
MAE	Mean Absolute Error. Average of absolute differences. Less sensitive to outliers.
R-squared	Proportion of variance explained by the model. 1.0 = perfect, 0 = baseline.
Derivative	Rate of change of a function at a point. 'How steep is the curve here?'
Gradient	Vector of all partial derivatives. Points in direction of steepest cost increase.
Gradient Descent	Algorithm: update weights by moving opposite to gradient, repeatedly.
Learning rate	Step size in gradient descent. Controls how large each weight update is.
Epoch	One complete pass through the training dataset.
Iteration	One weight update step. Mini-batch GD: many iterations per epoch.
Convergence	When the cost stops decreasing. Training is complete.
Divergence	When cost increases instead of decreasing. Learning rate is too large.

Batch GD	Gradient computed on entire dataset per update. Accurate, slow.
SGD	Stochastic GD. Gradient computed on one sample. Fast, noisy.
Mini-batch GD	Gradient computed on 32-256 samples. Default in deep learning.
Hyperparameter	A setting you choose before training. Learning rate, batch size, n_epochs.
Weight update rule	$w = w - \text{learning_rate} * \text{gradient}$. The core of gradient descent.

SECTION HW Homework — Submit Before Day 12

Create `day11_homework.py` and push to GitHub before midnight. This is the last homework of Phase 1. Day 12 is the EDA mini-project. Make sure you understand every concept from today because Phase 2 uses all of them.

Exercise 1 — Line and Slope (Beginner)

Given the line $y = 4.5x + 12$: (1) What does y equal when $x = 0, 5, 10, 15$? Print all four values. (2) What does the slope 4.5 mean in the context of predicting house price (y , in lakhs) from size (x , in hundreds of sqft)? Write your interpretation as a comment. (3) Plot the line using matplotlib over the range $x = 0$ to 20.

Exercise 2 — Cost Function Explorer (Beginner-Intermediate)

You have 6 data points: $x=[1,2,3,4,5,6]$, $y=[3,5,7,9,11,13]$. Write a function `compute_cost(slope, intercept, x, y)` that returns the MSE. Then compute and print the MSE for these five (slope, intercept) combinations: (2,1), (2,0), (2,2), (1.8,1.2), (2.05,0.95). Which gives the lowest MSE? Plot all five lines on one chart with the data points.

Exercise 3 — Gradient Descent Implementation (Intermediate)

Implement gradient descent from scratch (without using the class from today's notes). Start with $w=0$, $b=0$. Use `learning_rate=0.01` and run for 500 iterations on the dataset: $\text{hours}=[2,4,6,8,10]$, $\text{score}=[52,65,75,85,95]$. Print w , b , and MSE every 100 iterations. Plot the cost history as a line chart showing how cost decreases over iterations.

Exercise 4 — Learning Rate Experiment (Intermediate)

Using the same dataset from Exercise 3, run gradient descent with five different learning rates: 0.0001, 0.001, 0.01, 0.05, 0.5. For each, track the cost every 10 iterations and store in a list. Create a single plot showing all five cost curves (learning rate as label). Identify which learning rates converge, which are too slow, and which diverge.

Exercise 5 — Multiple Features (Intermediate)

Generate synthetic data: 50 students with two features (study_hours: 1-10, sleep_hours: 5-9) and exam score = $5 \times \text{study_hours} + 3 \times \text{sleep_hours} + 20 + \text{noise}$. Implement gradient descent for multiple features (X is now a 50x2 matrix). Run for 2000 iterations. Report the learned weights and compare them to the true weights (5.0 and 3.0). How close did gradient descent get?

Stretch Challenge — Tier A students

Extend today's LinearRegressionFromScratch class to support: (1) Multiple features using matrix operations. (2) L2 Regularisation (Ridge): add $\lambda \times w^2$ to the cost and $2 \times \lambda \times w$ to the gradient. Test with lambda values 0, 0.1, 1.0, 10.0 on a dataset with 5 features and only 20 training samples. Show how regularisation prevents overfitting by comparing training vs validation MSE for each lambda. Plot the results as a line chart.